

Sesión 03
Líneas de Productos Software

Anass El Jabiry Kaddir

Departamento de Informática
Universidad de Almería

Almería, lunes 11 de noviembre de 2024

Índice de Contenidos

Página

1. Introducción	3
2. Desarrollo Paso a Paso	3
Paso 1: Creación del Modelo de Datos	3
Paso 2: Creación de la Vista de Lista de Amigos (VistaListaAmigos.swift)	5
Paso 3: Implementación del Filtrado de Favoritos.....	6
Paso 4: Navegación entre Vistas	7
Paso 5: Modificación del Estado de Favorito.....	8
Paso 6: Edición de Opiniones sobre los Amigos.....	9
3. Ejercicios Realizados	9
Ejercicio 1: Reemplazar amigoCurrent por índice del Array	9
Ejercicio 2: Personalización del Fondo del TextEditor	10
Ejercicio 3: Implementación de la Barra de Progreso	10
Ejercicio 4: Comprobación de la Persistencia de Cambios.....	11
4. Conclusiones y Reflexiones	11

1. Introducción

El objetivo de esta tercera práctica fue añadir nuevas funcionalidades a la aplicación **GestionAmigos**, en la que se implementó una gestión de lista de amigos, se introdujo la navegación entre vistas y se permitió la modificación de ciertos datos de los amigos. En esta sesión trabajamos principalmente con la presentación de una tabla de amigos y la navegación hacia una vista de detalle de cada amigo, utilizando elementos como List, NavigationView, Toggle y otros componentes visuales de SwiftUI.

El desarrollo de la aplicación incluyó funcionalidades como el filtrado de amigos favoritos y la edición de opiniones sobre los mismos, lo que enriquecieron la experiencia de usuario. Este informe documenta el proceso y los pasos seguidos para lograr los objetivos propuestos, además de responder a preguntas específicas que surgieron durante la práctica.

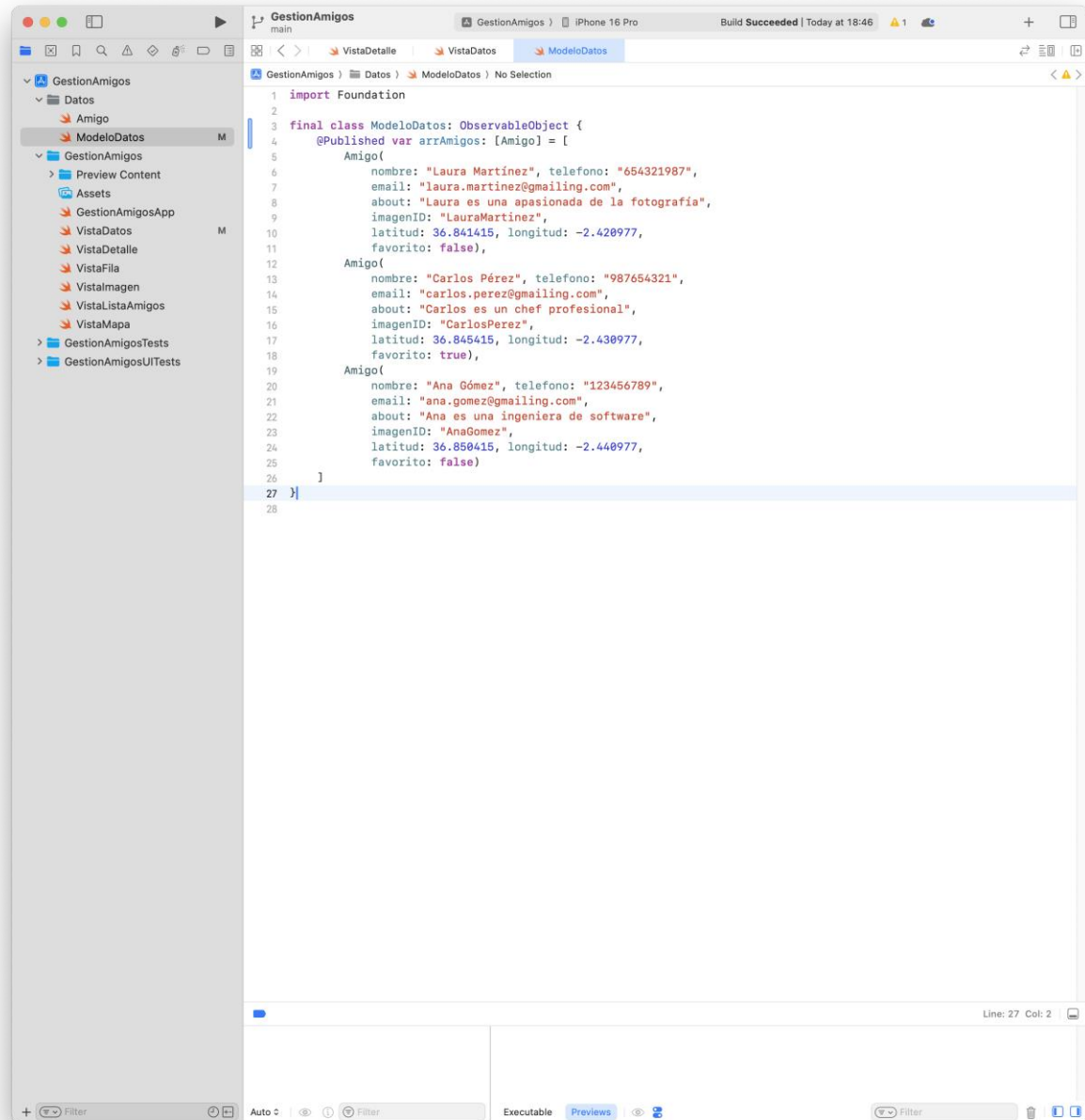
Herramientas Utilizadas

- **Xcode 13.4.1:** Entorno de desarrollo integrado utilizado para programar, depurar y ejecutar la aplicación.
- **Swift:** Lenguaje de programación empleado, explorando nuevas características y conceptos como @EnvironmentObject y publicación de datos compartidos.
- **SwiftUI:** Framework que facilitó la declaración de la interfaz y la creación de vistas interactivas y adaptativas.

2. Desarrollo Paso a Paso

Paso 1: Creación del Modelo de Datos

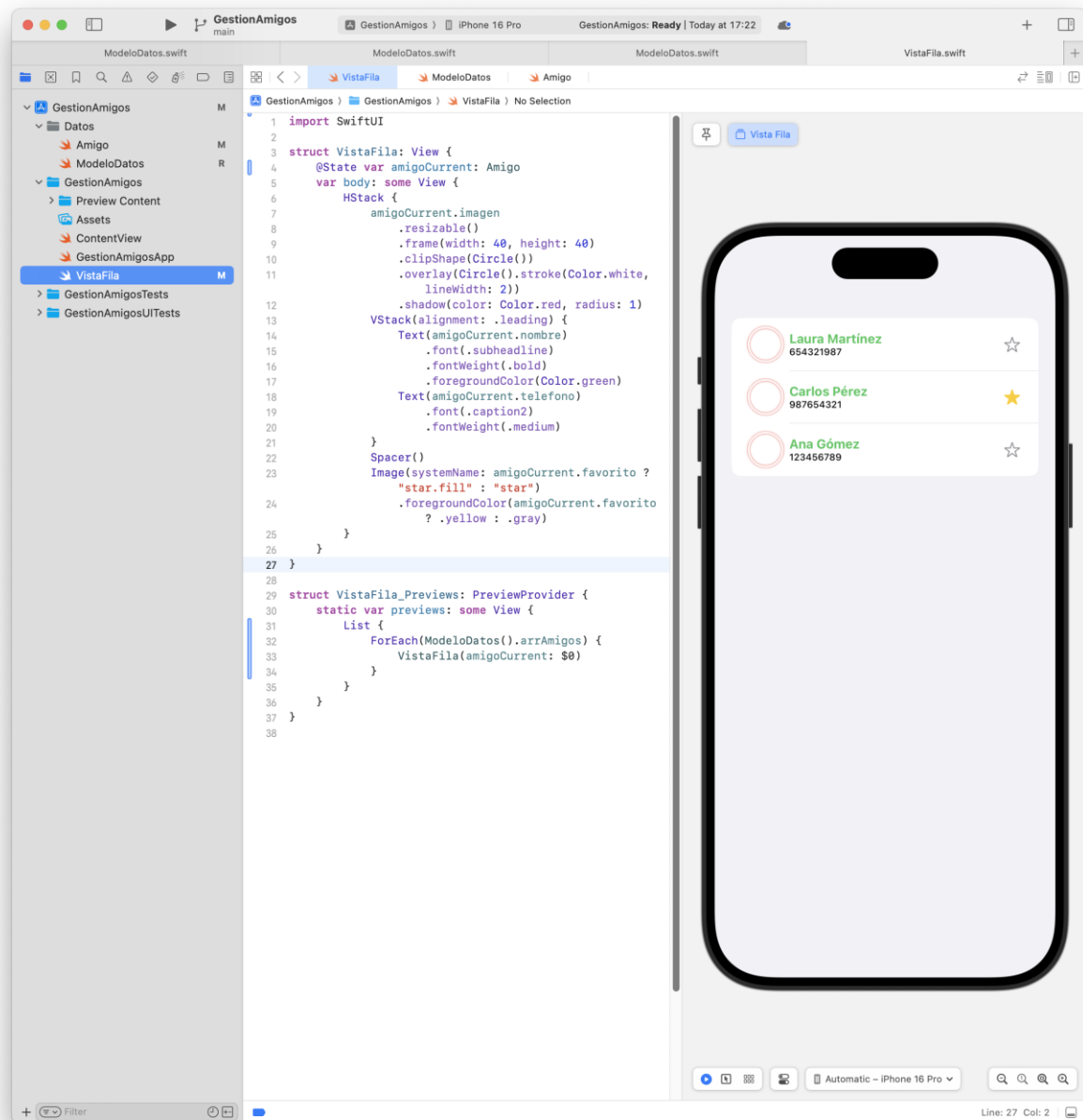
El primer paso fue crear el modelo de datos de la aplicación. Para ello, definimos la estructura **Amigo** dentro de un archivo llamado Amigo.swift, la cual incluye atributos como nombre, telefono, email, imagenID, latitud, longitud y un nuevo atributo favorito. Además, se creó una clase llamada ModeloDatos que encapsula un array de instancias de Amigo denominado arrAmigos.



Captura 1: Código de la estructura Amigo y el array de amigos inicial.

Paso 2: Creación de la Vista de Lista de Amigos (VistaListaAmigos.swift)

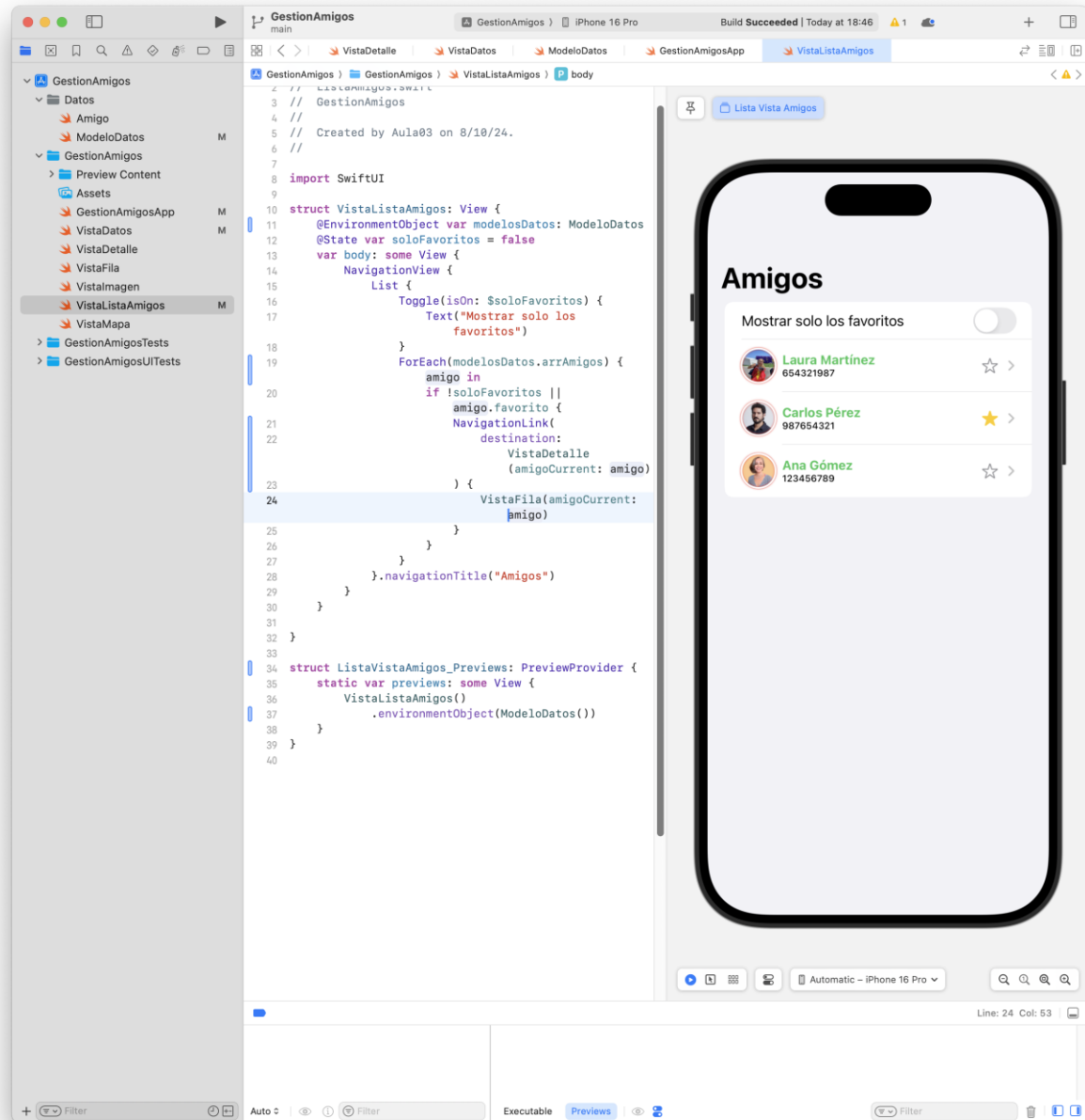
En este paso se desarrolló la vista VistaListaAmigos, la cual utiliza una lista (List) para mostrar a todos los amigos que contiene arrAmigos. Para cada amigo, se generó una fila personalizada que se definió en una subvista llamada VistaFila.



Captura 2: La vista de lista de amigos mostrando varias filas con sus correspondientes datos y estrellas que indican si es un amigo favorito o no.

Paso 3: Implementación del Filtrado de Favoritos

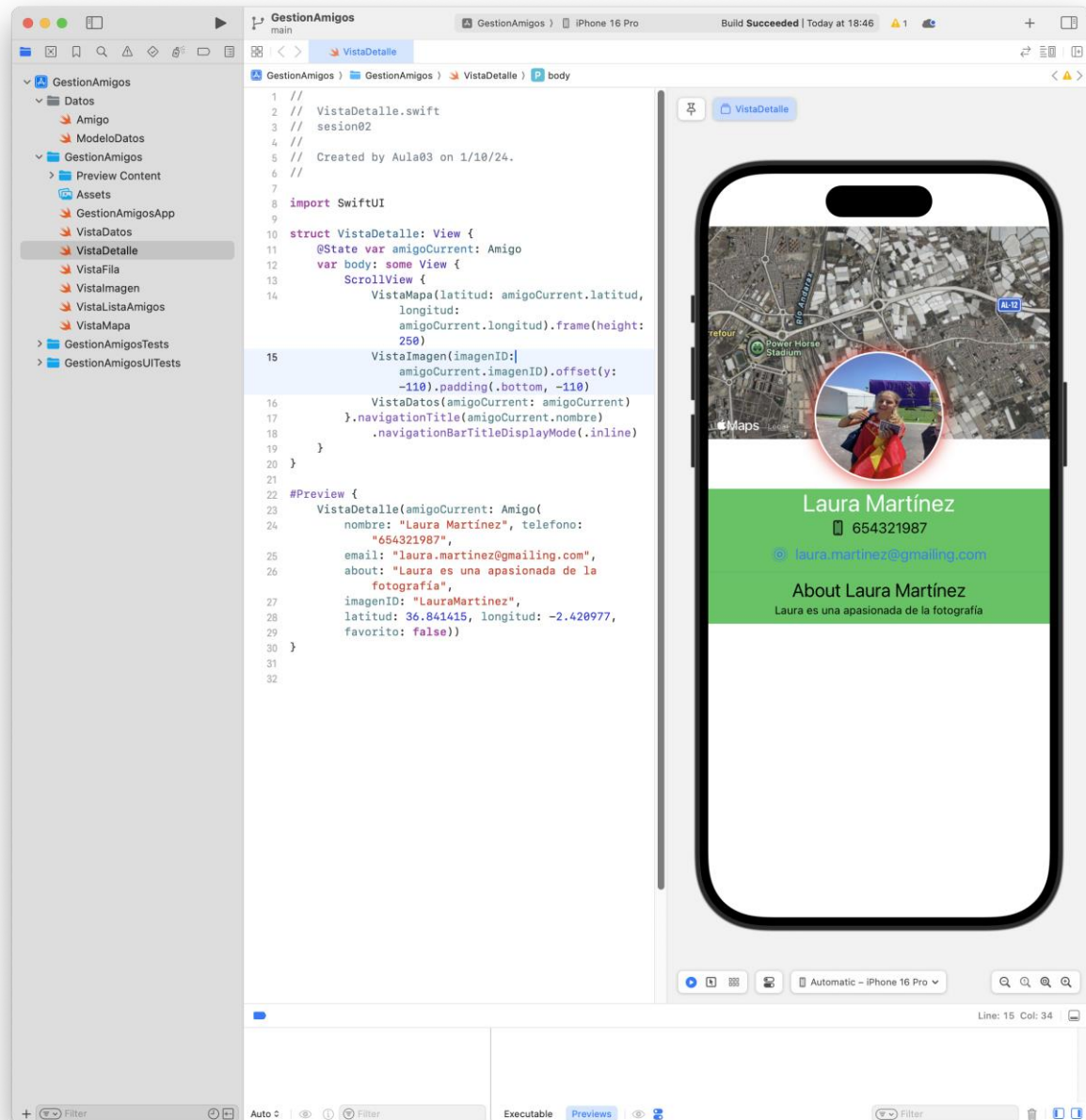
Se agregó la capacidad de filtrar la lista para mostrar sólo los amigos que han sido marcados como favoritos. Utilizamos un Toggle para activar o desactivar este filtro, y creamos una variable de estado `@State var soloFavoritos` para controlar el estado del filtro. El componente `List` se combinó con un `ForEach` que iteraba sobre los amigos y aplicaba la condición del filtro.



Captura 3: VistaListaAmigos mostrará solo los amigos favoritos cuando el Toggle está activado.

Paso 4: Navegación entre Vistas

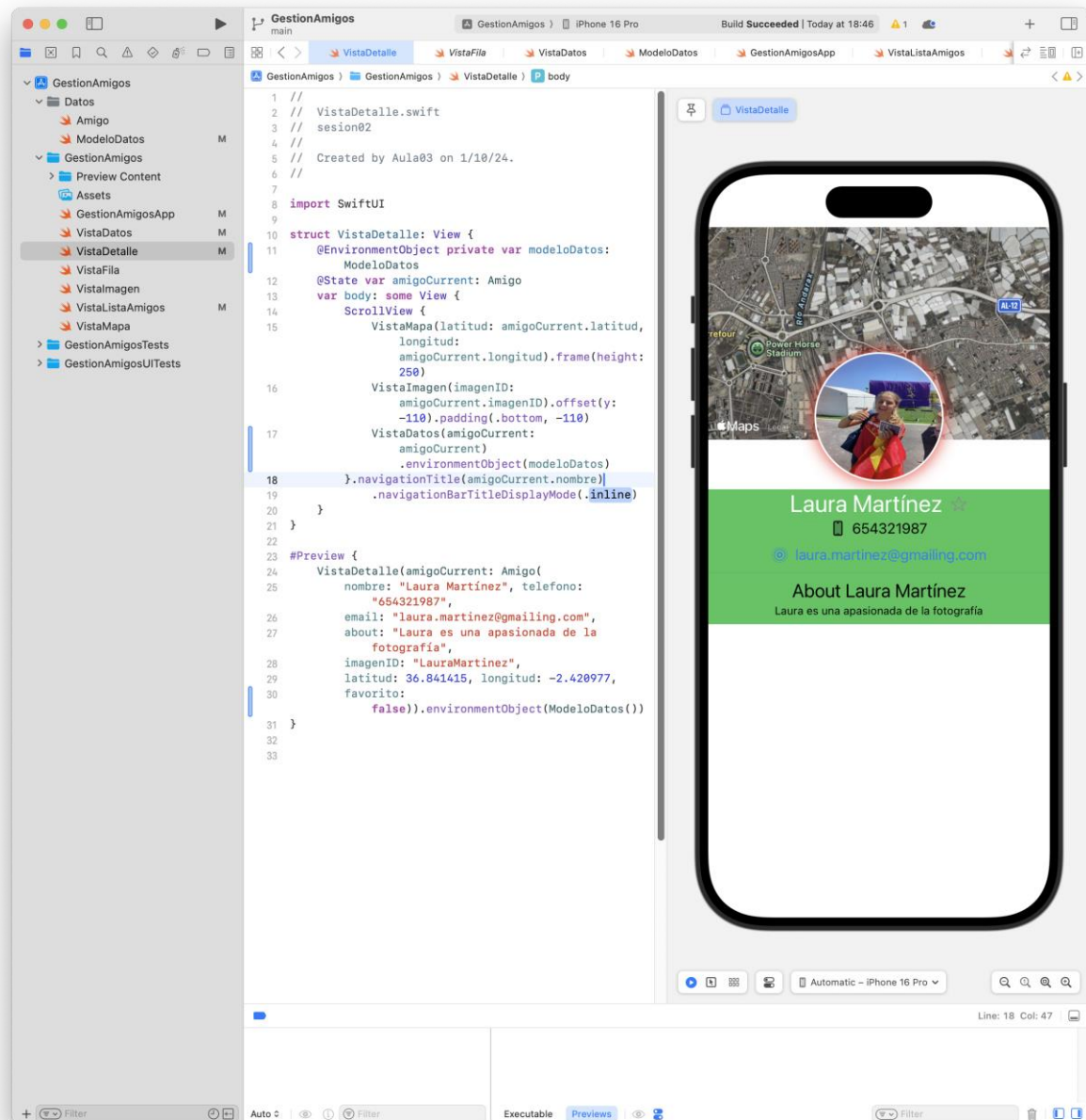
Para permitir la navegación entre la lista de amigos y los detalles de un amigo en particular, se utilizó el componente `NavigationView` junto con `NavLink`. Cada fila en la lista estaba enlazada a la vista de detalle (`VistaDetalle`) correspondiente al amigo seleccionado. Además, se parametrizó `VistaDetalle` para recibir un objeto `Amigo` como argumento, lo cual permite mostrar la información específica del amigo.



Captura 4: Navegación desde la lista de amigos hacia la vista de detalle de un amigo.

Paso 5: Modificación del Estado de Favorito

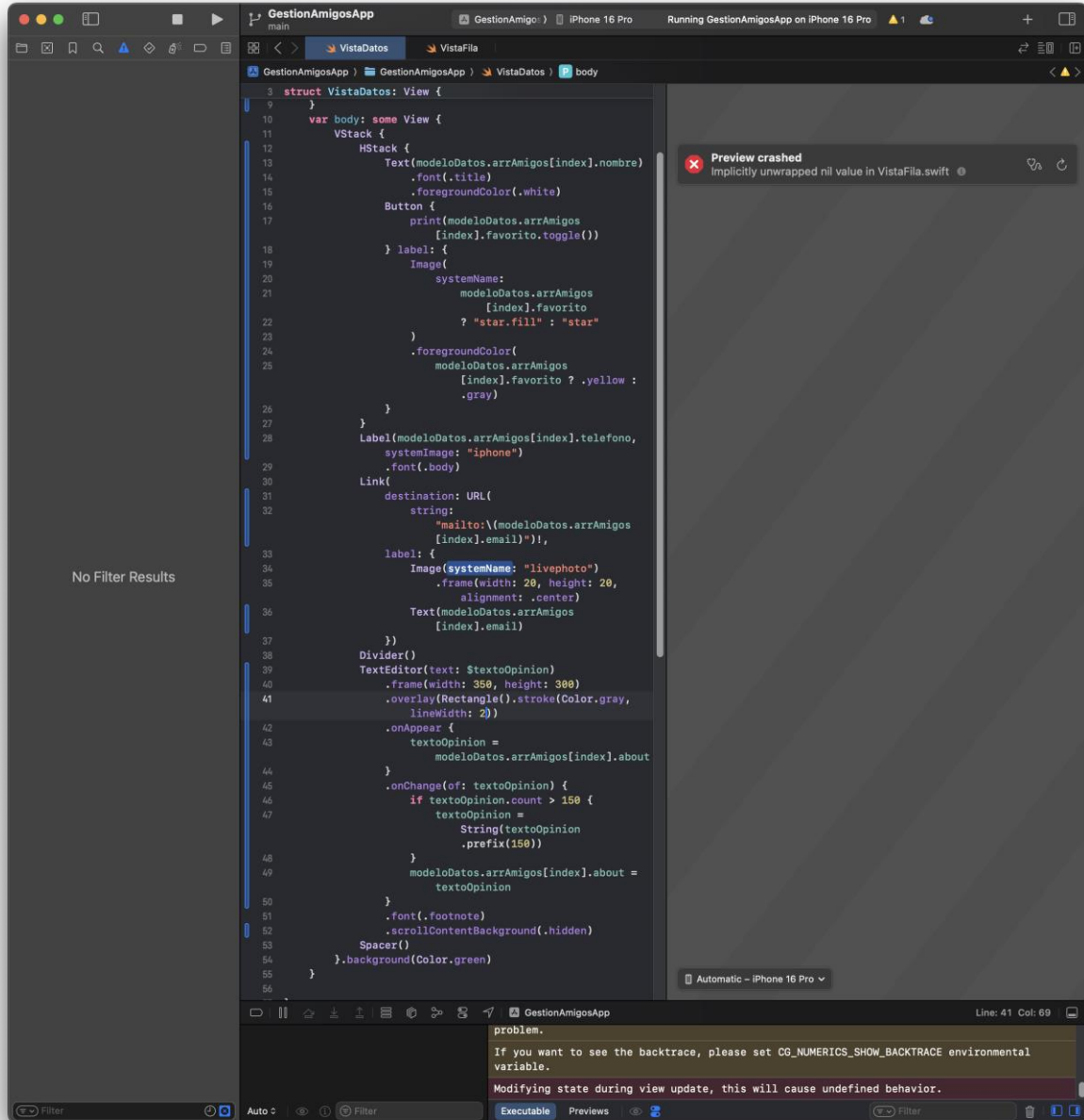
En la vista de detalle (VistaDetalle), se implementó la capacidad de cambiar el estado del atributo favorito mediante un botón. Para que estos cambios fueran reflejados en toda la aplicación, se necesitó que el modelo de datos fuera compartido entre vistas. Para lograr esto, la clase ModeloDatos se hizo conforme al protocolo ObservableObject y se publicó la lista arrAmigos con el atributo @Published. Además, se utilizó @EnvironmentObject en cada vista que necesitaba acceso al modelo.



Captura 5: Botón para marcar o desmarcar a un amigo como favorito en la vista de detalle.

Paso 6: Edición de Opiniones sobre los Amigos

Se añadió un campo de texto editable (TextEditor) en VistaDatos que permite al usuario escribir una opinión sobre el amigo. Para mejorar la experiencia del usuario, se limitó el número de caracteres a 150 e incorporamos una ProgressView para mostrar la cantidad de caracteres escritos. Cuando se superaban los 100 caracteres, el texto cambiaba de color para advertir al usuario.



Captura 6: Campo de opinión editable con barra de progreso mostrando la cantidad de caracteres escritos.

3. Ejercicios Realizados

Ejercicio 1: Reemplazar amigoCurrent por índice del Array

Se planteó la posibilidad de eliminar la variable amigoCurrent y, en su lugar, pasar la posición del array donde se encuentra el amigo que estamos editando. Esto haría que la implementación fuera más flexible, ya que facilitaría el uso de otros tipos de estructuras, como diccionarios. La solución consiste en pasar el índice del amigo al constructor de las vistas y luego acceder a los datos directamente desde el modelo ModeloDatos usando ese índice.

- **Implementación Alternativa:**

```

struct VistaDetalle: View {
    @State var amigoCurrent: Amigo
    var body: some View {
        ScrollView {
            VistaMapa(latitud: amigoCurrent.latitud,
                      longitud:
                        amigoCurrent.longitud).frame(height:
                          250)
            VistaImagen(imagenID:|
                        amigoCurrent.imagenID).offset(y:
                          -110).padding(.bottom, -110)
            VistaDatos(amigoCurrent: amigoCurrent)
        }.navigationTitle(amigoCurrent.nombre)
          .navigationBarTitleDisplayMode(.inline)
    }
}

```

Ejercicio 2: Personalización del Fondo del TextEditor

Para que el fondo del TextEditor coincidiera con el color de fondo de la vista, simplemente se aplicó el modificador `.background(Color.green)`. De esta manera, se mantuvo la coherencia visual en toda la aplicación.

Ejercicio 3: Implementación de la Barra de Progreso

Se implementó la barra de progreso para mostrar la cantidad de caracteres que el usuario había ingresado en el TextEditor. Para evitar que se excediera el límite de 150 caracteres, se utilizó el evento `.onChange(of:)` asociado a la variable del texto para eliminar cualquier exceso de caracteres de manera interactiva.

- **Código:**

```

ProgressView(
    "Número de caracteres:
      \(characterCount)/150",
    | value: min(Double(characterCount), 150),
      total: 150
)
.progressViewStyle(LinearProgressViewStyle())
.accentColor(characterCount >= 100 ? .red :
              .blue)
.frame(width: 300)
Spacer()

```

Ejercicio 4: Comprobación de la Persistencia de Cambios

Para verificar que los cambios realizados en el campo de texto se guardaban correctamente en el modelo de datos, se utilizó el evento `.onDisappear` para almacenar los cambios en el atributo `about` del amigo antes de que la vista se cerrara. Este evento se asoció al `VStack` principal de la vista `VistaDetalle` para asegurar que cualquier cambio hecho se actualizara correctamente en el modelo de datos.

- **Código:**

```
.onDisappear {  
    modeloDatos.arrAmigos[index].about = textoOpinion  
}
```

4. Conclusiones y Reflexiones

En esta práctica avanzamos significativamente en la complejidad de la aplicación, aprendiendo a manejar listas y a permitir la navegación entre diferentes vistas. También profundizamos en la utilización de datos compartidos entre vistas utilizando el protocolo `ObservableObject` y en cómo hacer modificaciones que se propaguen en toda la aplicación.

Las funcionalidades de filtrado, navegación y edición hicieron que la aplicación fuera más interactiva y permitieron que los usuarios tuvieran un mayor control sobre los datos, lo cual resultó en una experiencia más rica y completa.